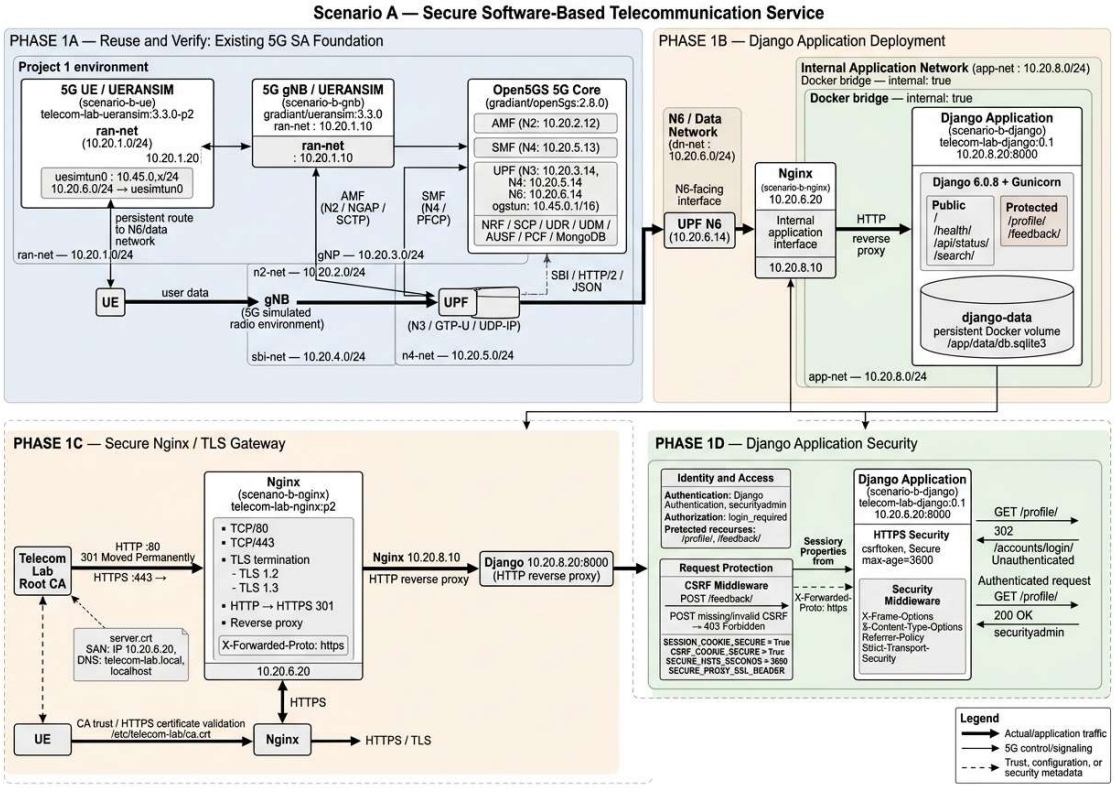
Project 2: Telecommunication Security, DevSecOps, and Traffic Intelligence.

Project 2 extends the software-based telecommunications environment established in Project 1 by adding security to the same reproducible Docker-based system. The existing 5G Standalone network, including UERANSIM, Open5GS, MongoDB, the N6 data network, Prometheus, Grafana, cAdvisor, iperf3, and the existing Docker networks, remains the foundation. Project 2 progressively adds application security, security testing, DevSecOps automation, network security monitoring, and traffic intelligence. The overall engineering workflow is Reuse → Secure → Model → Test → Detect → Automate → Analyze → Defend. Project 2 is divided into two levels. Scenario A — BASIC establishes a secure and reproducible software-based telecommunications service. Scenario B — ADVANCED extends this service with security testing, DevSecOps automation, intrusion detection, 5G traffic/security visibility, and security-oriented traffic intelligence.

Scenario A: Secure Software-Based Telecommunication Service

1. Scenario Overview: Scenario A reuses the operational Project 1 5G Standalone network and extends the N6/data-network side with a secure application service. The main additions are an enhanced Nginx reverse proxy and TLS gateway and a containerized Django web application. The purpose of Scenario A is to establish a complete security baseline in which an application carried over the 5G user plane is protected by HTTPS/TLS, separated through Docker network segmentation, protected by authentication and authorization, defended against CSRF, and configured with secure cookies and HSTS. The progression is: Phase 1A — Reuse and Verify → Phase 1B — Deploy Django → Phase 1C — Secure the Gateway with Nginx and TLS → Phase 1D — Establish the Django Security Baseline. At the end of Phase 1D, the application must operate successfully through the real 5G path with verified HTTPS, certificate validation, authentication, access control, CSRF protection, secure cookies, HSTS, and security headers. Phase 1E is outside the core construction of Scenario A. It begins the security-testing stage and provides the transition toward Scenario B.

2. Reference Architecture: The reference user-plane path is: UE → gNB → UPF → N6 → Nginx → Django. Nginx is connected to both dn-net and app-net. Django is connected only to app-net. Consequently, Django is not directly attached to the N6/data network. The N6-facing gateway therefore provides a clear application-security boundary between the telecommunications data plane and the internal application service. The 5G interfaces and Docker networks must remain conceptually distinct. For example, N3 is a 3GPP interface carrying GTP-U, whereas n3-net is the Docker network used to implement the corresponding connectivity in this laboratory environment. The UPF tunnel interface is ogstun = 10.45.0.1/16. The persistent UE route required by the application path is: 10.20.6.0/24 → uesimtun0. This route allows traffic destined for the N6/data-network subnet to enter the 5G PDU tunnel rather than using the ordinary Docker ran-net route. A recommended baseline topology is shown below (take it as a reference!)



3. Software and Images: Existing Project 1 Foundation: The following components are retained and are not rebuilt as new Project 2 functions: UERANSIM 3.3.0, Open5GS 2.8.0, MongoDB, iperf3, Prometheus, Grafana, cAdvisor, Docker Engine, Docker Compose, ran-net, n2-net, n3-net, sbi-net, n4-net, dn-net and obs-net. These provide the operational 5G, data-plane, and observability foundation. New Scenario A Images: Scenario A introduces three locally built images.

Nginx: Image: telecom-lab-nginx:p2; Base image: nginx:latest; Additional software: iproute2. nginx:latest is the new base image required for the Nginx build. It is extended locally to provide persistent routing and TLS-related configuration. Main functions: reverse proxy, TLS termination, HTTPS on TCP/443, HTTP on TCP/80, HTTP → HTTPS redirect, X-Forwarded-Proto forwarding, N6/data-network gateway, routing support for the UE PDU path, certificate mounting,

Django: Image: telecom-lab-django:0.1. Base image: python:3.12-slim; Application: Django 6.0.8; Application server: Gunicorn; Database: SQLite. python:3.12-slim is the new base image required for the Django build. Main functions: web application, authentication, authorization, CSRF protection, secure session handling, application API, user-input processing, health checking.

UERANSIM UE: Image: telecom-lab-ueransim:3.3.0-p2; Base image: gradiant/ueransim:3.3.0. This is a locally built extension of the existing UERANSIM image. No new 5G UE implementation is introduced. The custom image adds a startup mechanism that maintains the route: 10.20.6.0/24 → uesimtun0. This makes the Scenario A data path reproducible after UE container recreation.

Verification-only Images: Two additional images may be used temporarily for testing but are not part of the Scenario A runtime architecture: curlimages/curl:latest and alpine. They are used for HTTPS, certificate, and TLS handshake verification and are removed after each test with docker run --rm. Scenario A requires several files to be created or updated manually in the repository. The main structure is shown as follows. Private credentials and private TLS keys must not be committed to Git. The repository excludes: env; secrets/tls/. The laboratory CA private key remains on the host and is never mounted into the UE, Nginx, or Django containers.



Step 1: Reuse Project 1 Network and Deploy the Django Application

Phase 1A establishes the starting condition for Scenario A: the Project 1 telecommunications network is operational, and the 5G user-plane path reaches the N6/data network. The existing UE, gNB, Open5GS Core, UPF, N6 network, and monitoring environment remain unchanged except for the persistent-route support subsequently added to the UE image. The verification must cover UE registration, PDU session establishment, the presence of uesimtun0, the dynamically assigned 10.45.0.x address, and the route toward the N6 subnet. The required route is: 10.20.6.0/24 dev uesimtun0. A route lookup for the Nginx address must resolve through the PDU interface: 10.20.6.20 dev uesimtun0. The end-to-end baseline must also demonstrate reachability from the UE to Nginx. Phase 1A checkpoint: The Project 1 5G network is considered ready when the UE can reach 10.20.6.20 through uesimtun0.

Phase 1B introduces the application layer. The Django application is built as a custom container using: python:3.12-slim; Django 6.0.8; Gunicorn. The container is: scenario-b-django (10.20.8.20:8000) and is connected only to: app-net (10.20.8.0/24). The application must not be directly connected to dn-net. The first application endpoint is: /health/ and returns:{"service": "project2-django","status": "ok","project": "Telecommunication Security"}. The application is configured using environment variables, including the Django secret key, debug mode, and allowed hosts. A Django health check is configured so that Docker can determine whether the application is operational. The SQLite database is stored at /app/data/db.sqlite3 and is backed by the named Docker volume: django-data. The database must survive Django container recreation. Database initialization is completed with Django migrations before creating the laboratory authentication account. Required application deployment condition: Django image builds → python manage.py check→Django container becomes healthy→Nginx can reach Django→/health/ returns HTTP 200. Required evidence: Dockerfile, requirements.txt, Compose service, healthcheck, container health status, app-net configuration, django-data volume, and /health/ response.

Step 2: Configure Nginx as the Secure Application Gateway

Phase 1C introduces the application gateway between the N6/data network and the internal Django service. Nginx is represented by: scenario-b-nginx, image: telecom-lab-nginx:p2 and has two network interfaces: dn-net → 10.20.6.20 and app-net → 10.20.8.10. Django remains: app-net → 10.20.8.20. The application path therefore becomes: UE→5G PDU Session→UPF→N6→Nginx 10.20.6.20→app-net→Django 10.20.8.20:8000. Nginx provides: TCP/80 for HTTP; TCP/443 for HTTPS; TLS termination; TLS 1.2; TLS 1.3; reverse proxying; HTTP → HTTPS redirection; forwarding of X-Forwarded-Proto: https. The Nginx-to-Django connection remains inside the isolated internal application network. A local laboratory PKI is created for the application gateway. The certificate hierarchy is: Telecom Lab Root CA→server certificate→Nginx 10.20.6.20. The server certificate must contain: IP:10.20.6.20 and may additionally contain: DNS:telecom-lab.local; DNS:localhost. The server certificate and private key are mounted read-only into Nginx. The CA certificate is mounted read-only into the UE: /etc/telecom-lab/ca.crt. The CA private key must remain outside the containers. The final HTTPS test must perform genuine certificate validation: curl --cacert /etc/telecom-lab/ca.crt ... The final service verification must not depend on: curl -k ... // -k may be used during initial TLS troubleshooting, but it is not accepted as the final certificate-validation demonstration. Required TLS evidence: Certificate subject; Certificate issuer; Validity period; Subject Alternative Name; TLS 1.2 handshake; TLS 1.3 handshake; and UE HTTPS request with trusted CA. After certificate and TLS verification, HTTP access is changed from normal application delivery to an HTTPS redirect. The expected behavior is: HTTP:80→301 Moved Permanently→HTTPS:443→Django

application. The complete test must demonstrate: HTTP request→ 301 redirect→ HTTPS request→ HTTP 200 response. The HTTPS service becomes the normal application transport.

Step 3: Build the Django Security Baseline

Phase 1D turns the basic Django health service into a small, security-testable web application. The application exposes:/; /health/; /api/status/; /search/; /accounts/login/; /accounts/logout/; /profile/; /feedback/; /admin/. The endpoints are divided into public and protected resources. Public resources include: /; /health/; /api/status/; /search/. Protected resources include /profile/; /feedback/. The protected resources use Django authentication and authorization mechanisms. A controlled laboratory account is created through Django: securityadmin. The security model distinguishes two concepts. Authentication determines whether the requester has a valid identity. Authorization determines whether that authenticated identity is permitted to access a protected resource. The required behavior is: Unauthenticated request→GET /profile/ →302→/accounts/login/. After successful authentication: Authenticated request→GET /profile/→200 OK →Username: securityadmin/Authenticated: True. The same authorization principle applies to /feedback/. The feedback application contains an authenticated POST form protected by Django's CSRF middleware. The normal successful path is: Authenticated request→POST /feedback/→valid CSRF token→200 OK. The negative security test removes the CSRF token: Authenticated request → POST /feedback → missing/invalid CSRF token→403 Forbidden. The rejection must remain enabled. CSRF protection must not be disabled merely to make a test request succeed. The two tests establish both normal application behavior and explicit enforcement of a security control. Django is configured so that authentication-related cookies are protected for HTTPS use. Required settings: SESSION_COOKIE_SECURE = True; CSRF_COOKIE_SECURE = True. The resulting session cookie must demonstrate the expected secure attributes, including: Secure; HttpOnly; SameSite=Lax. The CSRF cookie must contain: Secure. The purpose of this configuration is to prevent authentication and CSRF cookies from being transmitted over ordinary unencrypted HTTP.

HSTS is enabled as a short-duration laboratory baseline: SECURE_HSTS_SECONDS = 3600. The following settings remain disabled for the laboratory: SECURE_HSTS_INCLUDE_SUBDOMAINS = False and SECURE_HSTS_PRELOAD = False. The HTTPS response must contain: Strict-Transport-Security: max-age=3600. HSTS strengthens HTTPS-only browser behavior; it does not replace TLS. TLS terminates at Nginx, while Nginx communicates with Django through the isolated app-net. Django therefore needs to recognize the original client request as HTTPS. The required setting is: SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https"). Nginx sends:X-Forwarded-Proto: https. The resulting architecture is: Client / UE→through HTTPS→Nginx→Django, where HTTP internally and X-Forwarded-Proto: https. This setting does not change the network path. It allows Django's security logic to correctly interpret the original client connection after TLS termination at the reverse proxy.

Django's security middleware must remain enabled. The application response is expected to contain security-related headers such as: X-Frame-Options; X-Content-Type-Options; Referrer-Policy; Strict-Transport-Security. The actual header values must be recorded as evidence. Security should be demonstrated through observable HTTP behavior rather than described only as a configuration claim. The current reference deployment produces, among others: X-Frame-Options: DENY; X-Content-Type-Options: nosniff; Referrer-Policy: same-origin; Strict-Transport-Security: max-age=3600. The final application demonstration must use the actual 5G user-plane path rather than connecting directly to Django. The required path is: UE→5G PDU Session→gNB→N3 / GTP-U→UPF→N6→Nginx 10.20.6.20→HTTPS / TLS → app-net → Django 10.20.8.20:8000. Direct access to 10.20.8.20:8000 is useful for internal troubleshooting, but it is not the final end-to-end demonstration. The following functional checks form the Scenario A acceptance test:

UE → Nginx
HTTP → 301 → HTTPS
HTTPS + trusted CA → 200 OK
GET /
GET /health/
GET /api/status/

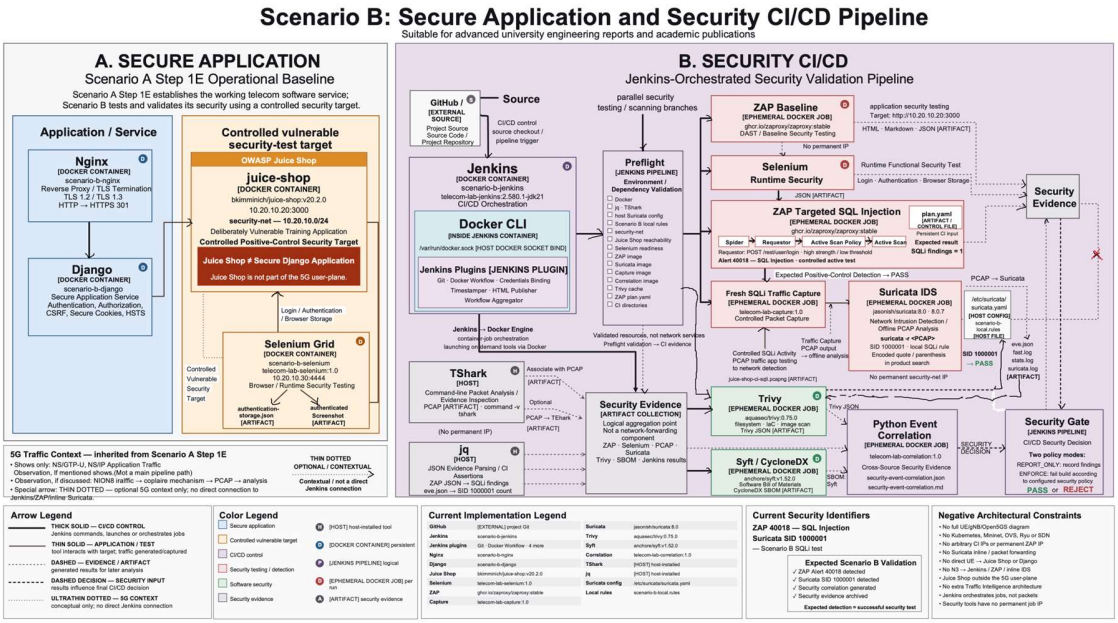
GET /search/
Authentication and access control:
Unauthenticated /profile/ → 302
Unauthenticated /feedback/ → 302
Authenticated /profile/ → 200
Authenticated /feedback/ → 200
Valid CSRF POST → 200
Missing/invalid CSRF → 403
Security headers and cookies:
Strict-Transport-Security
Secure CSRF cookie
Secure session cookie
HttpOnly session cookie
X-Frame-Options
X-Content-Type-Options
Referrer-Policy

Scenario B: Advanced Security CI/CD and Security Validation

Scenario A unmentioned phase 1e is the required transition point into Scenario B. At the end of Scenario A, you must have already demonstrated that the secure application operates successfully through the 5G user-plane path and that the Nginx/Django service satisfies the established TLS, authentication, authorization, CSRF, secure-cookie, HSTS, and security-header requirements. Phase 1e therefore does not introduce another application or security tool; it establishes the verified operational baseline from which Scenario B begins. The Scenario B environment is an additional security-engineering layer around that working system. Its purpose is to move from a manually verified secure service toward a reproducible security workflow in which security tests are automatically executed, network evidence is captured, intrusion detection is performed, software artifacts are assessed, results are correlated, and a CI/CD security decision is produced. The overall progression is: Validate the baseline → Test the application → Generate controlled security traffic → Detect the traffic → Analyze traffic intelligence → Assess software artifacts → Correlate evidence → Gate the build. The main practical security target used in Scenario B is the deliberately vulnerable OWASP Juice Shop, which is isolated from the secure Scenario A Django/Nginx application and is used as a controlled positive-control target rather than as a production service.

Scenario B reference architecture. Scenario B extends the secure application baseline established in Scenario A Phase 1e and adds a dedicated Security CI/CD environment around it. The secure application remains the Scenario A scenario-b-nginx reverse proxy/TLS gateway and scenario-b-django internal application, while Scenario B introduces an isolated, controlled security-testing environment using security-net — 10.20.10.0/24. The main persistent security-test services are OWASP Juice Shop at 10.20.10.20:3000 and Selenium Grid at 10.20.10.30:4444. Juice Shop is the deliberately vulnerable training application and controlled positive-control target; it is intentionally separated from the secure Django/Nginx application and is not part of the 5G user-plane. Selenium provides browser-level runtime interaction with Juice Shop, including automated login, authentication-state verification, browser storage inspection, cookie inspection, and screenshot evidence. Jenkins is the central CI/CD orchestration point and controls the security workflow rather than acting as a telecom network function. The important architecture is therefore a set of parallel but connected paths: project source/GitHub → Jenkins → Preflight → security jobs; Jenkins → Selenium → Juice Shop; Jenkins → OWASP ZAP → Juice Shop; controlled SQLi activity → Traffic Capture → PCAP → Suricata; and ZAP results + Selenium evidence + PCAP + Suricata results + Trivy results + SBOM + Jenkins results → Security Evidence → Python Correlation → Security Gate. ZAP baseline testing provides automated application discovery and passive security assessment, while the targeted ZAP test performs a controlled SQL-injection validation against Juice Shop. The targeted test uses a persistent plan.yaml, seeds a controlled POST /rest/user/login request, and enables ZAP alert 40018 — SQL Injection; the expected positive-control result is a detected SQLi finding. The resulting traffic is captured as a fresh PCAP and subsequently analyzed by Suricata, creating a second, network-level security-detection path. This architecture should be presented as a security-engineering extension of

Scenario A rather than as a second 5G network, and security tools must not be depicted as 5G functions or packet-forwarding nodes. The original architecture likewise defined Scenario B as an extension of the working Scenario A system and organized it around DevSecOps, security testing, and later evidence processing. A recommended baseline topology is shown below (take it as a reference!)



Scenario B software, containers, tools, and implementation model. The central orchestrator is scenario-b-jenkins, built from telecom-lab-jenkins:2.580.1-jdk21, with Jenkins plugins including Git, Docker Workflow, Credentials Binding, Timestamper, HTML Publisher, and Workflow Aggregator; these plugins are internal Jenkins components rather than separate network nodes. Jenkins has the Docker CLI and access to /var/run/docker.sock, allowing the pipeline to launch security tools as controlled Docker jobs. The persistent controlled services are juice-shop, using bkimminich/juice-shop:v20.2.0 at 10.20.10.20:3000, and Selenium Grid, using telecom-lab-selenium:1.0 at 10.20.10.30:4444. OWASP ZAP uses ghcr.io/zaproxy/zaproxy:stable and is launched as an ephemeral Docker job for both baseline and targeted SQLi testing. The targeted SQLi workflow uses plan.yaml, a ZAP requestor for POST /rest/user/login, and active-scan rule 40018; the current working pipeline verifies SQL Injection findings = 1. Traffic Capture uses telecom-lab-capture:1.0 as an ephemeral Docker job and generates juice-shop-ci-sqli.pcapng. Suricata uses jasonish/suricata:8.0 and is executed as an ephemeral offline-PCAP analysis job using the Suricata binary directly, the host configuration /etc/suricata/suricata.yaml, and security/ids/rules/scenario-b-local.rules; the current detected local rule is SID 1000001, LOCAL Scenario B SQLi test - encoded quote/parenthesis in product search. The resulting evidence includes eve.json, fast.log, stats.log, and suricata.log. Trivy uses aquasec/trivy:0.75.0 as an on-demand vulnerability/security-scanning job, while Syft uses anchore/syft:v1.52.0 to generate the CycloneDX SBOM. Python correlation uses telecom-lab-correlation:1.0 to combine the ZAP, Selenium, PCAP, Suricata, Trivy, SBOM, and Jenkins evidence into security-event-correlation.json and security-event-correlation.md. On the host, jq is used for JSON parsing and CI assertions and TShark is used for command-line PCAP inspection; /etc/suricata/suricata.yaml and the local IDS rule file remain host-side inputs and are mounted read-only into the Suricata analysis job. The current implementation deliberately does not require SonarQube or Snyk, and it does not assign artificial persistent IP addresses to ZAP, Suricata, Trivy, Syft, Traffic Capture, or Python correlation because these are launched on demand by Jenkins.

Security CI/CD — Jenkins-Orchestrated Security Validation Pipeline：

Scenario B is delivered to students as a pre-integrated, reproducible security-testing environment. Students are not expected to install individual security tools manually. The delivered environment provides vulnerable applications, security-testing containers, Jenkins orchestration, network-capture capability,

analysis utilities, and evidence directories required for the complete workflow. Jenkins is the central orchestrator, but each component has a clearly defined responsibility. The pipeline must be executed in a deterministic order because later stages depend on artifacts, services, or evidence generated by earlier stages. All externally supplied software components must be version-pinned in the Scenario B Release Manifest. The Release Manifest must identify the exact image tag or package version for ZAP, Selenium/Grid/browser components, Suricata, Jenkins, the Jenkins plugin set, the capture image, Trivy, Syft, TShark, jq, and the correlation image. A student should never have to guess which software version the course reference environment expects. Jenkins plugins must also be treated as part of the reproducible environment. Plugins required by the Jenkinsfile must be pre-installed in the delivered Jenkins image and their versions recorded by Preflight. Plugins used only for presentation, such as enhanced Jenkins visualization, must not be hidden dependencies for security-test correctness. Scenario B used component and execution contract:

Layer	Component / Tool	Execution Environment	Primary Function	Primary Evidence
Application	OWASP Juice Shop	Controlled application container	Deliberately vulnerable security-test target	Application reachability and runtime state
Orchestration	Jenkins	Jenkins container	Executes and coordinates all stages	Jenkins console, stage results, exit codes
Web Security	OWASP ZAP	ZAP container	Baseline, targeted, and active application-security testing	HTML, Markdown, JSON, plan.yaml, Alert 40018
Browser Runtime	Selenium Grid + browser node	Selenium containers	Authentication/session/storage/runtime validation	JSON assertions and screenshot
Traffic Evidence	telecom-lab-capture:1.0	Dedicated capture environment	Generates a fresh PCAP for the controlled test run	juice-shop-ci-sqli.pcapng
Network IDS	Suricata	Suricata container	Offline detection against the fresh PCAP	eve.json, fast.log, stats.log, suricata.log
Packet Analysis	TShark	CI analysis environment	Offline packet inspection and evidence verification	Packet/request assertions
Structured Parsing	jq	CI analysis environment	JSON extraction and assertions	Parsed ZAP/Suricata values
Vulnerability Scanner	aquasec/trivy:0.75.0	On-demand CI container	Filesystem, IaC, and/or image security scanning according to the configured pipeline	Trivy report
SBOM	anchore/syft:v1.52.0	On-demand CI container	Software composition inventory	CycloneDX SBOM
Evidence Correlation	telecom-lab-correlation:1.0	Correlation container	Correlates independent security observations	JSON + Markdown correlation reports
Security Decision	Jenkins Security Gate	Jenkins pipeline	Converts configured findings into a CI/CD decision	

Phase 2A: Preflight and Environment Acceptance

Phase 2A is a hard dependency gate. No security test should begin until Preflight confirms that the complete Scenario B environment is ready. Preflight verifies the Scenario A Step 1E operational baseline and then validates the Scenario B execution environment. The checks must be deterministic and must produce machine-readable and human-readable evidence. Required Preflight checks: Jenkins execution environment; Jenkins is running. The Jenkinsfile can execute. Required Jenkins plugins are installed. Required plugin versions match the Scenario B Release Manifest. The Jenkins workspace is writable. Container/runtime environment. Docker is reachable from the Jenkins execution environment according to the delivered architecture. Required Scenario B images are available or can be pulled from the configured registry. Required Docker networks exist. Container DNS/service discovery works where used. Application availability: Juice Shop is running. http://10.20.10.20:3000 is reachable from the security-testing environment. A basic application health/reachability check succeeds. The pipeline confirms that the target is the controlled Scenario B Juice Shop instance, not an unintended external target. Selenium readiness: Selenium Grid is reachable. A browser session can be created. The expected browser/node is ready to execute the Scenario B runtime test. ZAP readiness:

The ZAP container is running. The expected ZAP API/interface is reachable. The target URL is configured correctly. The persistent plan.yaml exists and is syntactically valid for the installed ZAP Automation Framework. The targeted SQLi policy configuration is available. Traffic-capture readiness: The capture environment exists. The required capture interface is visible. The capture process has the permissions required to observe packets. The selected capture point can actually observe the Juice Shop HTTP traffic generated by the test. The output directory is writable. Merely starting a packet-capture container is not sufficient evidence of readiness. Preflight must demonstrate that the selected capture point can observe the intended traffic path. Suricata readiness: The Suricata executable is available. The Scenario B configuration is available. The local rule file is present. SID 1000001 is loaded by the active rule set. The Suricata output directory is writable. Analysis-tool readiness: jq is available. TShark is available. Python is available for the correlation stage. Required scripts are present and executable. Software-security tooling: aquasec/trivy:0.75.0 is available. anchore/syft:v1.52.0 is available. Required databases/configuration for the configured Trivy mode are accessible. Evidence hygiene: A new RUN_ID is created. The current evidence directory is empty or uses a run-specific namespace. Old PCAPs, reports, JSON files, and logs cannot be mistaken for current-run artifacts. The pipeline records the timestamp and tool versions used for the run. Preflight decision: Preflight PASS means the environment is technically ready. Preflight FAIL means downstream security testing must not proceed. A security result must never be interpreted when the environment itself has not passed Preflight.

Phase 2B: Application Security Testing

Phase 2B validates application security through complementary testing paths. The paths have different purposes and therefore produce different types of evidence.

OWASP ZAP Baseline Testing:

The ZAP container performs the configured baseline scan against:
http://10.20.10.20:3000. The baseline procedure performs the configured discovery/spidering and passive security analysis and produces the required HTML, Markdown, and JSON reports.

Baseline findings are security observations, not automatically build failures. A warning or informational finding becomes a CI/CD failure only when the configured Security Gate explicitly assigns that condition a failure policy. Required evidence: ZAP baseline HTML report; ZAP baseline Markdown report; ZAP baseline JSON report. Jenkins stage result and exit status. The baseline stage establishes the security state of the application before the controlled positive-control test.

Selenium Runtime and Authentication Validation:

Selenium Grid interacts with Juice Shop through a real browser session. The purpose of this stage is not to replace ZAP or Suricata. It validates application behavior that is best observed from the browser/runtime layer, including: login behavior; authentication state; session state; cookies; browser storage; authenticated application state; required assertions about the resulting runtime state. The test produces structured JSON evidence and an authenticated screenshot. Selenium therefore provides runtime/application-behavior evidence, while ZAP provides automated application-security analysis. Required evidence: Selenium JSON result; Authentication/session assertions; Browser storage/cookie evidence required by the test; Authenticated screenshot and Jenkins stage result.

Arm the Fresh Network-Capture Window:

Before the controlled SQLi activity begins, Jenkins starts a new capture window. This ordering is mandatory. The pipeline must not generate the SQLi traffic first and capture it afterward, because doing so may produce a valid PCAP that simply contains none of the intended SQLi traffic. The capture window must cover the complete controlled activity and be associated with the current RUN_ID. At minimum, the capture lifecycle is: START CAPTURE → GENERATE CONTROLLED SQLi ACTIVITY → COMPLETE TARGETED TEST → STOP CAPTURE. The resulting PCAP is therefore tied to the actual current execution.

Controlled SQLi Network Positive Control:

The pipeline generates the deterministic HTTP transaction used by the Scenario B network-security positive control. The current reference local Suricata rule is: SID: 1000001. Description: LOCAL Scenario B SQLi test - encoded quote/parenthesis in product search. The current reference request pattern is: GET /rest/products/search?q=%27%28 directed to: 10.20.10.20:3000. This request must be deliberately generated against the controlled Juice Shop target while the fresh packet capture is active. The guidebook

must explicitly distinguish this deterministic network positive control from the ZAP Alert 40018 test. They may be coordinated within the same test run, but they are not automatically the same HTTP transaction.

Targeted ZAP SQL Injection Test:

The targeted ZAP test is the principal application-security positive control. The pipeline uses the persistent plan.yaml Automation Framework configuration; performs the configured short discovery/spidering step; creates or loads the dedicated active-scan policy; sends the controlled request through the ZAP requestor as configured; executes the targeted active scan; verifies the configured SQL Injection rule.

The current reference policy enables:

ZAP Alert 40018 — SQL Injection with: high strength; low threshold. The current reference result is: SQL Injection findings = 1. This is an expected positive-control result. Therefore: Alert 40018 detected → PASS for the positive-control test. You cannot interpret the deliberate detection of this controlled vulnerability as "the application is secure." The purpose is to prove that the security-testing mechanism is working correctly against a known vulnerable target. Required evidence: Persistent plan.yaml; Copy of the executed plan.yaml associated with the current RUN_ID; Targeted ZAP JSON report; Verified Alert 40018 finding; Jenkins command/exit-code evidence. The configuration file proves what was requested. The ZAP result proves what was actually detected.

Phase 2C: Controlled Security Traffic, PCAP Generation, and Network IDS Detection

Phase 2C converts the controlled application-security activity into independently inspectable network evidence. The capture itself was armed before the Step 2B controlled SQLi activity. After the activity finishes, Jenkins stops the capture and verifies that a fresh PCAP has been created. The current reference artifact is: juice-shop-ci-sqli.pcapng. The pipeline must verify that: the PCAP belongs to the current RUN_ID; its creation time falls inside the current test window; the file is non-empty; the expected Juice Shop traffic is present; it is not an artifact left from a previous run. TShark is used for offline packet inspection and jq is used for structured result assertions.

Suricata PCAP analysis: The Suricata container analyzes the fresh PCAP offline using the host-mounted Scenario B configuration and local rule set. Conceptually: PCAP → Suricata offline analysis → eve.json / fast.log / stats.log / suricata.log. This is not inline IDS forwarding and does not require Suricata to be inserted into the live application forwarding path. The local rule uses: SID 1000001 — LOCAL Scenario B SQLi test - encoded quote/parenthesis in product search. The CI pipeline parses the JSON-lines eve.json and verifies that at least one event contains SID 1000001. Expected positive-control result: SID 1000001 detected → PASS. ZAP Alert 40018 is an application-security finding produced by ZAP. Suricata SID 1000001 is a network-detection rule match produced by Suricata. They are different identifiers, different tools, and different detection layers. Required evidence: Fresh juice-shop-ci-sqli.pcapng; Suricata configuration; Scenario B local rule; eve.json; fast.log; stats.log; suricata.log; TShark verification; Jenkins verification that SID 1000001 occurs at least once.

****Phase 2C-Advanced: Traffic Intelligence and Security Analytics Extension.**** After the fresh PCAP has been validated and Suricata has independently analyzed the controlled SQLi traffic, Scenario B may extend the same evidence pipeline with a bounded Traffic Intelligence function rather than introducing a separate traffic-intelligence architecture. The purpose of this extension is to demonstrate how packet-level and flow-level observations can be transformed into security-relevant features and analyzed using deterministic statistics, anomaly detection, machine-learning classification, or prediction methods. The input should be the current-run PCAP together with derived observations such as TShark-extracted flow records, packet counts, byte counts, flow duration, packet-rate statistics, protocol or HTTP attributes where available, and Suricata `eve.json` events; the analytics stage must not require Jenkins to receive N3 traffic directly and must not replace Suricata as the deterministic IDS mechanism. A reference implementation may use Python-based feature extraction and a lightweight model or anomaly detector to produce artifacts such as `traffic-features.csv`, `traffic-anomaly.json`, `traffic-classification.json`, or `traffic-prediction.json`. Example functions include identifying traffic patterns that deviate from a learned baseline, classifying controlled traffic into predefined categories, estimating the probability that a flow belongs to a known attack class, or predicting a future security-related condition when an appropriate labeled or temporal dataset is provided. Because Scenario B is primarily a security-validation exercise, these analytics results should initially be treated as **supplementary evidence** and should be correlated with the deterministic ZAP, PCAP, and Suricata results rather than being used as an unconditional build-failure trigger. The

resulting analytics artifacts are passed into the existing **Security Evidence → Python Security Event Correlation → Security Gate** path, allowing students to demonstrate the progression from **packet capture → feature extraction → anomaly detection / ML analysis → evidence correlation → policy decision** without creating a separate large “Traffic Intelligence” system. Where no sufficiently large or representative dataset exists, the guidebook should require a bounded demonstration using the supplied controlled traffic and clearly identify model output as an experimental or analytical result rather than production-grade detection accuracy.

Phase 2D: Software Vulnerability Scanning and Software Bill of Materials
Phase 2D evaluates the software artifacts and dependencies supporting the application.
Trivy: Trivy is executed as an on-demand CI container using aquasec/trivy:0.75.0. The exact scan modes are defined by the delivered Jenkinsfile and Release Manifest. The configured checks may include: filesystem security scanning; infrastructure-as-code scanning; container-image vulnerability scanning. The guidebook must identify explicitly which target is scanned in each command. The Trivy stage produces a structured security report. A Trivy finding and a Jenkins build failure are not the same thing. For example: Trivy finding = security observation while Security Gate decision = CI/CD policy decision; Syft / CycloneDX SBOM; Syft is executed using: anchore/syft:v1.52.0. The pipeline generates a CycloneDX-compatible Software Bill of Materials. The SBOM answers: What software components and dependencies are present? Trivy answers: Which configured security problems are identified in those artifacts? The SBOM and vulnerability report are therefore complementary rather than duplicate outputs. Required evidence: Trivy report; CycloneDX; SBOM; Jenkins stage and exit-code evidence.

Phase 2E Security Evidence Correlation, Evidence Archive, and Security Gate
Phase 2E converts independently generated observations into a reproducible security conclusion. The pipeline collects: ZAP baseline JSON/HTML/Markdown; Selenium JSON evidence; Selenium screenshot; targeted ZAP plan.yaml; targeted ZAP JSON; verified ZAP Alert 40018; fresh SQLi PCAP; Suricata eve.json; Suricata fast.log; Suricata stats.log; Suricata suricata.log; verified Suricata SID 1000001; Trivy report; CycloneDX SBOM; TShark verification; jq verification; Jenkins stage results and exit codes. These artifacts constitute the Scenario B Security Evidence Set. The correlation container, telecom-lab-correlation:1.0, receives the relevant evidence and produces: security-event-correlation.json, security-event-correlation.md. Correlation should connect evidence using explicit attributes such as: RUN_ID; execution time window; target address and port; request/traffic signature; security-tool result identifiers; artifact filenames; artifact hashes where implemented. The correlation process must not invent evidence. Its purpose is to demonstrate how independently generated observations are related. A representative correlation chain is: Controlled SQLi activity → application-security observation in ZAP → corresponding network traffic in PCAP → Suricata network detection → evidence correlation → CI/CD security decision. TShark and jq support this stage: TShark provides packet-level verification. jq provides structured extraction and machine-readable assertions. Python correlation combines the independent observations. Security Gate: The Security Gate is a Jenkins policy decision, not another scanner. The pipeline supports:
REPORT_ONLY: Security findings are recorded and archived. The configured Trivy HIGH/CRITICAL condition does not, by itself, fail the build. ENFORCE: The configured security policy is applied as a CI/CD gate. When the defined condition is violated, Jenkins rejects the build. The guidebook must always report these separately: Tool execution result → Security finding → Gate decision. For example: Trivy completed successfully → HIGH vulnerability exists → REPORT_ONLY = build continues, or Trivy completed successfully → HIGH/CRITICAL threshold violated → ENFORCE = build fails. Similarly, the controlled positive controls have their own expected results: ZAP 40018 detected → positive-control PASS; Suricata SID 1000001 detected → positive-control PASS. A successful pipeline therefore does not mean that the deliberately vulnerable application contains no vulnerabilities. It means that the configured security-validation process executed correctly and produced the expected evidence and policy decision.

The Jenkins pipeline should make the dependency order explicit:
1. Scenario A Step 1E acceptance
↓
2. 2A Preflight

↓
3. 2B-1 ZAP Baseline
↓
4. 2B-2 Selenium Runtime/Auth Validation
↓
5. 2B-3 Start Fresh Packet Capture
↓
6. 2B-4 Generate Controlled SQLi Network Positive Control
↓
7. 2B-5 Execute Targeted ZAP SQLi Test
↓
8. 2C Stop Capture and Validate Fresh PCAP
↓
9. 2C Suricata Offline PCAP Analysis
↓
10. 2C-Advanced Traffic Intelligence / Analytics (optional)
↓
11. 2D Trivy Security Scan
↓
12. 2D Syft CycloneDX SBOM Generation
↓
13. 2E Security Evidence Correlation
↓
14. Evidence Archive
↓
15. Security Gate
↓
16. Final Jenkins Result
The pipeline must not silently reorder these dependencies.

Summary:
Students must extend the reproducible software-based telecommunications environment established in Project 1 into a secure and testable telecommunications system and demonstrate how security can be validated, monitored, analyzed, and automated. For Scenario A, students must reuse the working 5G Standalone network and establish the secure application path UE → gNB → N3/GTP-U → UPF → N6 → Nginx → Django, with verified TLS/HTTPS, authentication, authorization, CSRF protection, secure cookies, HSTS, security headers, and Docker network segmentation. For Scenario B, students must extend this baseline with a reproducible Jenkins-orchestrated Security CI/CD pipeline that performs Preflight validation, ZAP application-security testing, Selenium runtime testing, controlled SQLi testing, fresh PCAP capture, offline Suricata detection, Trivy security scanning, and Syft/CycloneDX SBOM generation. Students must collect and correlate application, packet, IDS, software, and CI evidence using TShark, jq, and Python, and produce a configurable Security Gate with REPORT_ONLY and ENFORCE modes. The Traffic Intelligence function extends the same PCAP/evidence path and may include feature extraction, anomaly detection, machine-learning classification, or bounded prediction; these results are treated as supplementary security evidence rather than a replacement for deterministic ZAP/Suricata validation. Students must demonstrate at least one complete evidence chain from controlled security activity → traffic/application observation → detection or analysis → evidence → correlation → security decision, and push all implementation materials, configurations, scripts, pipeline definitions, reports, and analysis code to GitHub. The README should make clear how the system was secured → tested → detected → analyzed → automated → defended, together with deployment, verification, evidence, reproducibility, and known limitations.